Database System Management

Project Report

Ahmad Paturusi

TABLE OF CONTENTS

1 WHAT WAS DONE IN THE PROJECT

This report documents all changes made to the dbsm_projectdb database as part of the Database Systems Management course project. The database contains data for a company, covering information about their employees, customers, projects, departments, headquarters, skills, user groups, and geographic locations.

The following were created/implemented for the given tasks:

- Four views
- Three triggers
- Three stored procedures
- Three access roles with different permission levels
- Structural database changes

2 VIEWS

Four views were created to give management easier access to combined information from multiple tables. All four views are in views.sql.

2.1 view_employee_overview

Joins employee, job_title, department, headquarters, and geo_location to give a complete set of info regarding a certain employee. Management can see each employee's role, department, location, salary, and how it compares to the base salary of their job title.

```
-- view 1: employee overview w/ department, job title, and location
CREATE OR REPLACE VIEW view_employee_overview AS
SELECT
    e.e_id, e.emp_name, e.email, e.contract_type, e.salary,
    j.title AS job_title, j.base_salary,
    d.dep_name AS department,
    hq.hq_name AS headquarters,
    gl.city AS city, gl.country AS country
FROM employee e
LEFT JOIN job_title j ON e.j_id = j.j_id
LEFT JOIN department d ON e.d_id = d.d_id
LEFT JOIN headquarters hq ON d.hid = hq.h_id
LEFT JOIN geo_location gl ON hq.l_id = gl.l_id;
```

2.2 view_project_overview

Joins project, customer, and geo_location to show all projects with client info and country. Useful for tracking which customer is behind each project as well as where they're located.

```
-- view 2: project overview w/ customer and location info
CREATE OR REPLACE VIEW view_project_overview AS
SELECT
    p.p_id, p.project_name, p.budget, p.commission_percentage,
    p.p_start_date, p.p_end_date,
    c.c_name AS customer_name, c.c_type AS customer_type,
    gl.city AS customer_city, gl.country AS customer_country
FROM project p
LEFT JOIN customer c ON p.c_id = c.c_id
LEFT JOIN geo_location gl ON c.l_id = gl.l_id;
```

2.3 view_employee_skills

Joins employee, employee_skills, and skills to show which skills each employee has and if those skills carry salary benefits. Management can also query this view further to find employees with specific skills.

```
-- view 3: employee skills w/ salary benefits
CREATE OR REPLACE VIEW view_employee_skills AS
SELECT
    e.e_id, e.emp_name, e.salary,
    s.skill, s.salary_benefit, s.salary_benefit_value
FROM employee e
INNER JOIN employee_skills es ON e.e_id = es.e_id
INNER JOIN skills s ON es.s_id = s.s_id;
```

2.4 view_project_staffing

Joins project_role, project, employee, and job_title to show which employees are assigned to which projects. Useful for management if they want to check who makes up a ceratain team for a project.

```
-- view 4: project staffing
CREATE OR REPLACE VIEW view_project_staffing AS
SELECT
    p.p_id, p.project_name, p.budget,
    e.e_id, e.emp_name, e.email,
    j.title AS job_title, pr.prole_start_date
FROM project_role pr
INNER JOIN project p ON pr.p_id = p.p_id
INNER JOIN employee e ON pr.e_id = e.e_id
LEFT JOIN job_title j ON e.j_id = j.j_id;
```

3 TRIGGERS

Three triggers were created to automate business logic and maintain data integrity. All triggers are in triggers.sql.

3.1 check_duplicate_skill (BEFORE INSERT on skills)

Before a new skill is inserted, this trigger checks whether a skill with the same name already exists If it does, it raises an exception and blocks the insert.

```
CREATE OR REPLACE FUNCTION check_duplicate_skill()
RETURNS TRIGGER AS $$
DECLARE
    existing_skill VARCHAR;
BEGIN
    SELECT skill INTO existing_skill FROM skills
    WHERE LOWER(skill) = LOWER(NEW.skill);
    IF existing_skill IS NOT NULL THEN
        RAISE EXCEPTION 'Skill "%" already exists.', NEW.skill;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_duplicate_skill BEFORE INSERT ON skills
FOR EACH ROW EXECUTE FUNCTION check_duplicate_skill();
```

3.2 assign_employees_to_project (AFTER INSERT on project)

After a new project is inserted, the trigger finds the country of the linked customer, then selects up to three employees whose headquarters are in that country and creates project_role entries for them. This basically automates the initial team assignment for new projects.

```
CREATE OR REPLACE FUNCTION assign_employees_to_project()
RETURNS TRIGGER AS $$
DECLARE
    customer_country VARCHAR;
    emp RECORD;
    assigned INT := 0;
BEGIN
    SELECT gl.country INTO customer_country FROM customer c
    INNER JOIN geo_location gl ON c.l_id = gl.l_id
    WHERE c.c_id = NEW.c_id;

    FOR emp IN
        SELECT e.e_id FROM employee e
        INNER JOIN department d ON e.d_id = d.d_id
        INNER JOIN headquarters hq ON d.hid = hq.h_id
        INNER JOIN geo_location gl ON hq.l_id = gl.l_id
        WHERE gl.country = customer_country
          AND e.e_id NOT IN (
```

```
            SELECT e_id FROM project_role WHERE p_id = NEW.p_id)
        LIMIT 3
    LOOP
        INSERT INTO project_role (e_id, p_id, prole_start_date)
        VALUES (emp.e_id, NEW.p_id, CURRENT_DATE);
        assigned := assigned + 1;
    END LOOP;

    RAISE NOTICE '% employee(s) assigned to project %.', assigned,
NEW.p_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER assign_employees_to_project AFTER INSERT ON project
FOR EACH ROW EXECUTE FUNCTION assign_employees_to_project();
```

3.3 update_contract_dates (BEFORE UPDATE on employee)

When an employee's contract_type is changed, the trigger automatically sets contract_start to the current date. If the new type is Temporary, contract_end is set to two years from today. For any other type, contract_end is set to NULL because those contracts have no fixed end date.

```
CREATE OR REPLACE FUNCTION update_contract_dates()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.contract_type IS DISTINCT FROM OLD.contract_type THEN
        NEW.contract_start := CURRENT_DATE;
        IF NEW.contract_type = 'Temporary' THEN
            NEW.contract_end := CURRENT_DATE + INTERVAL '2 years';
        ELSE
            NEW.contract_end := NULL;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_contract_dates BEFORE UPDATE ON employee
FOR EACH ROW EXECUTE FUNCTION update_contract_date
```

4 PROCEDURES

Three procedures were created for common data management operations. All procedures are in procedures.sql and are called using the ”CALL” command.

4.1 set_salary_to_baselvl()

Sets every employee's salary to the base_salary that corresponds to their job title. Does a single UPDATE joining employee to job_title through the j_id foreign key. Call: CALL set_salary_to_baselvl();

```
CREATE OR REPLACE PROCEDURE set_salary_to_baselvl()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE employee SET salary = job_title.base_salary
    FROM job_title
    WHERE employee.j_id = job_title.j_id;
END;
$$;
```

4.2 increase_contracts()

Extends the contract_end date of all Temporary employees by 3 months. Only affects rows where contract_type is Temporary and contract_end is not NULL. Call: CALL increase_contracts();

```
CREATE OR REPLACE PROCEDURE increase_contracts()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE employee
    SET contract_end = contract_end + INTERVAL '3 months'
    WHERE contract_type = 'Temporary' AND contract_end IS NOT NULL;
END;
$$;
```

4.3 increase_salaries(percentage, salary_limit)

Increases salaries by a given percentage. It accepts an optional salary limit so that only employees earning below that value are affected. If the limit is NULL or 0, all employees get the raise. Examples: CALL increase_salaries(10); or CALL increase_salaries(5, 4000);

```
CREATE OR REPLACE PROCEDURE increase_salaries(
    p_percentage  NUMERIC,
    p_salary_limit INT DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE employee
    SET salary = ROUND(salary + (salary * p_percentage / 100))
    WHERE (p_salary_limit IS NULL OR p_salary_limit = 0)
       OR salary < p_salary_limit;
END;
$$;
```

5 ACCESS ROLES

Three roles were created with different levels of access. All role definitions are in roles.sql. Each role is created inside a DO block with exception handling. This makes it so that if the role already exists on the server, the creation of that role is skipped and the grants are still applied.

Role	Access Rights
admin	SUPERUSER (basically everything)
employee	CONNECT + SELECT on all tables in public schema
trainee	SELECT on project, customer, geo_location, project_role; SELECT on (e_id, emp_name, email) only from employee

```
DO $$ BEGIN
    CREATE ROLE admin WITH SUPERUSER LOGIN PASSWORD 'admin_password';
    EXCEPTION WHEN duplicate_object THEN
        RAISE NOTICE 'Role "admin" already exists, skipping.';
END $$;

DO $$ BEGIN
    CREATE ROLE employee WITH LOGIN PASSWORD 'employee_password';
    EXCEPTION WHEN duplicate_object THEN
        RAISE NOTICE 'Role "employee" already exists, skipping.';
END $$;

GRANT CONNECT ON DATABASE dbsm_projectdb TO employee;
GRANT USAGE ON SCHEMA public TO employee;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO employee;

DO $$ BEGIN
    CREATE ROLE trainee WITH LOGIN PASSWORD 'trainee_password';
    EXCEPTION WHEN duplicate_object THEN
        RAISE NOTICE 'Role "trainee" already exists, skipping.';
END $$;

GRANT CONNECT ON DATABASE dbsm_projectdb TO trainee;
GRANT USAGE ON SCHEMA public TO trainee;

GRANT SELECT ON project TO trainee;
GRANT SELECT ON customer TO trainee;
GRANT SELECT ON geo_location TO trainee;
GRANT SELECT ON project_role TO trainee;
GRANT SELECT (e_id, emp_name, email) ON employee TO trainee;
```

6 DATABASE CHANGES

The following structural changes were made to the database. All commands are in db_changes.sql.

6.1 zip_code column added to geo_location

A new VARCHAR column zip_code was added to the geo_location table using ALTER TABLE. No data was populated as the task did not require it.

6.2 NOT NULL constraints on customer email and project p_start_date

Before adding the NOT NULL constraints, existing NULL values were updated with placeholder values (placeholder@domain.com for email, CURRENT_DATE for p_start_date). This ensures the constraint could be applied without errors.

6.3 CHECK constraint on employee salary

A CHECK constraint was added to ensure employee salary is always greater than 1000. Any existing rows with salary at or below 1000 were first updated to 1001.

```
-- add zip_code column to geo_location
ALTER TABLE geo_location ADD COLUMN zip_code CHARACTER VARYING;

-- add NOT NULL to customer email
UPDATE customer SET email = 'placeholder@domain.com' WHERE email IS NULL;
ALTER TABLE customer ALTER COLUMN email SET NOT NULL;

-- add NOT NULL to project p_start_date
UPDATE project SET p_start_date = CURRENT_DATE WHERE p_start_date IS NULL;
ALTER TABLE project ALTER COLUMN p_start_date SET NOT NULL;

-- add CHECK constraint on employee salary
UPDATE employee SET salary = 1001 WHERE salary <= 1000;
ALTER TABLE employee ADD CONSTRAINT salary_check CHECK (salary > 1000);
```